

Model 1 vs Model 2

Fire & smoke detection on industrial cameras

Why a second model was built, how it was trained, and what it traded away

	Model 1	Model 2 (light)
Backbone	DINOv3 ViT-S/16 (21.6M)	DINOv3 ViT-Ti/16 (5.5M)
Detection head	Faster R-CNN (two-stage)	FCOS (one-stage)
Total parameters	39.3M	6.9M
Training	40 ep frozen + 15 ep partial unfreeze	45 ep frozen
Test mAP@0.5	0.7261	0.6125
Recall at 1% false-alarm rate*	0.899	0.831
Speed, same CPU	891 ms/frame	336 ms/frame (2.65x faster)
Batched export (multi-camera)	No - measured to fail	Yes - verified

** interpolated on each model's recall-versus-false-alarm curve; see section 5.*

Verdict. Model 2 is 2.7x faster, 5.7x smaller, and - unlike Model 1 - can be exported as a batched graph for many cameras. It paid for that with accuracy: 0.114 lower test mAP@0.5 and 6.8 points lower recall at the same 1% false-alarm rate. As trained, it is **not a drop-in replacement**; Model 1 remains the accuracy reference. Section 8 lists the two experiments that decide whether Model 2 can close the gap.

1. Why a second model was needed

The customer's target is **70 cameras**. At 10 frames per second that is 700 frames per second, and the standard way to serve that is to batch frames from many cameras through one GPU using an exported, optimised engine (ONNX, then TensorRT). Model 1 was measured to block that path:

Export attempt (Model 1)	Batch 1	Batch 4
Legacy ONNX exporter	works	fails in ONNX Runtime (batch size baked in)
torch.export (current exporter)	fails	-

The cause is structural. Faster R-CNN picks a *variable* number of candidate regions in the middle of the network and crops features for exactly those before its final layers run, so there is no fixed-shape point at which to cut the graph. A one-stage FCOS head places every learned layer before anything data-dependent, so it exports cleanly. The export of Model 2 was verified on every run: boxes decoded from the ONNX outputs match the PyTorch model to about 6e-5 pixels at batch sizes 1, 4 and 8.

A second, measured finding shaped the design: the head swap on its own did **not** make the model faster (ViT-S + FCOS ran at 0.96x). Roughly two-thirds of the compute is the DINOv3 trunk, so the speed had to come from a smaller DINOv3 - ViT-Ti. Input resolution was deliberately left at 640 px, because reducing it to 448 px had already been measured to cost 24 points of recall.

DINO was never removed. Both models use a DINOv3 backbone; Model 2 uses a smaller member of the same family, with a different head after it.

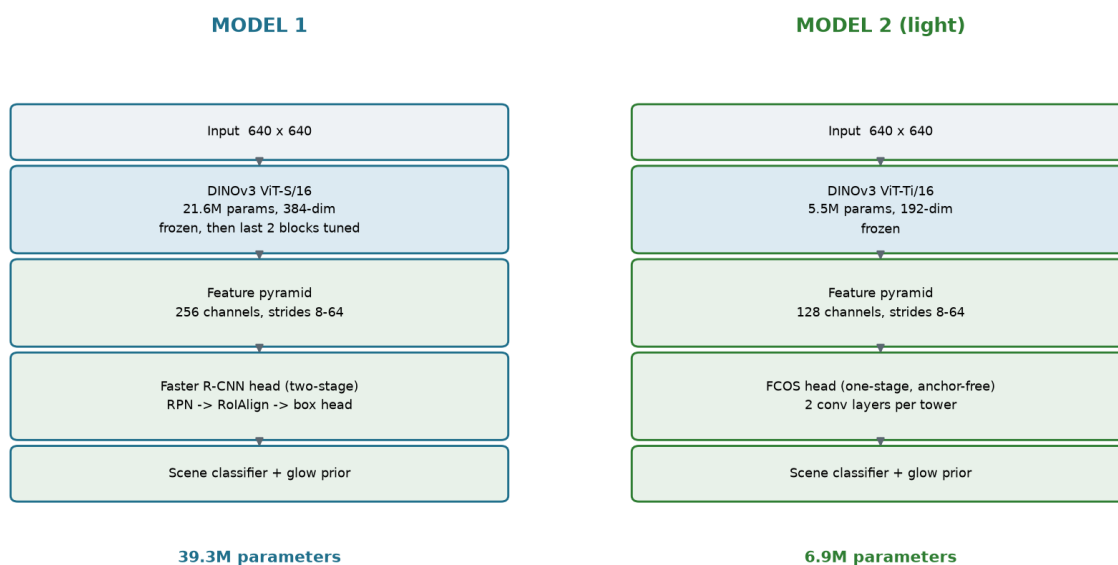


Figure 1 - The two architectures. Only the backbone size, pyramid width and detection head differ; input size, scene classifier and alarm logic are identical.

2. Configuration and training

Setting	Model 1	Model 2
Backbone	vit_small_patch16_dinov3 (LVD-1689M)	vit_tiny_patch16_dinov3_qkvb (EUPE)
Embedding dim / blocks	384 / 12	192 / 12
Pyramid channels	256	128
Head	Faster R-CNN, 300 test proposals	FCOS, 2 convs per tower
Trainable parameters	17.7M (stage 1)	1.5M
Input size / batch	640 / 8	640 / 8
Optimiser / LR	AdamW, 1e-4, cosine, 500-iter warmup	AdamW, 1e-4, cosine, 500-iter warmup
Augmentation	identical: night 0.35, IR 0.10, occlusion 0.25, ...	identical
Epochs	40 (stage 1) + 15 (stage 2)	45 (frozen trunk only)
Time per epoch	~11 min	~7.8 min
Best validation mAP@0.5	0.7406	0.6220

Both models were trained on the same D-Fire split with the same augmentation, optimiser and schedule, and selected by validation mAP@0.5, so the comparison isolates the architecture. One difference is not yet controlled: Model 1 received a second stage that unfroze its last two transformer blocks (worth +0.009 mAP), and Model 2 has not.

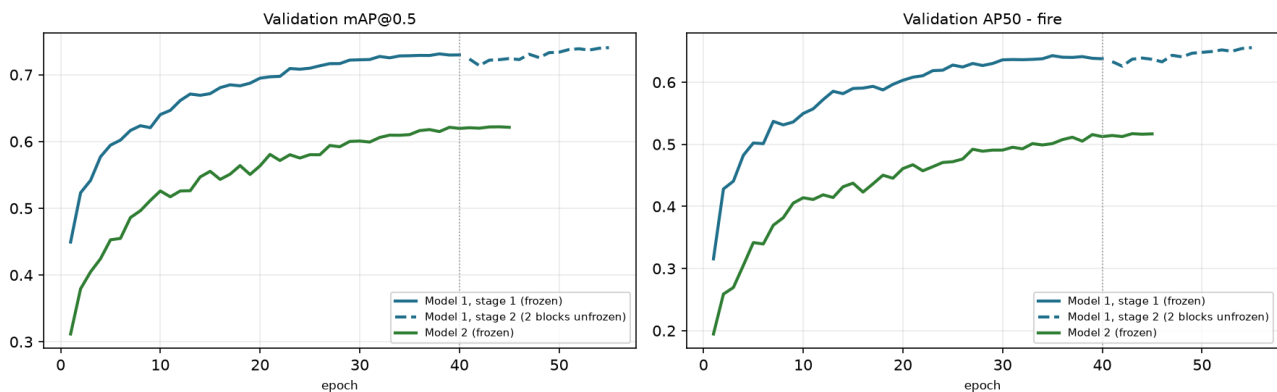


Figure 2 - Validation curves. Model 1's stage 2 continues from stage 1 (dashed, after the dotted line). Model 2 plateaus around 0.62 and was still inching upward at the end.

Model 2 converged smoothly but to a lower ceiling. With the trunk frozen it trains only 1.5M parameters against Model 1's 17.7M, which is the most likely single reason for the gap - and the reason a stage-2 run is the first thing to try.

3. Test-split results

Held-out D-Fire test split: 4,306 images, 2,005 of them verified negatives. Neither model saw it during training or model selection.

Class	Model 1 AP50	Model 2 AP50	Change	Model 1 AP75	Model 2 AP75
smoke	0.8146	0.7158	-0.0988	0.4053	0.3006
fire	0.6377	0.5092	-0.1285	0.2363	0.1404
mean	0.7261	0.6125	-0.1136	0.3208	0.2205

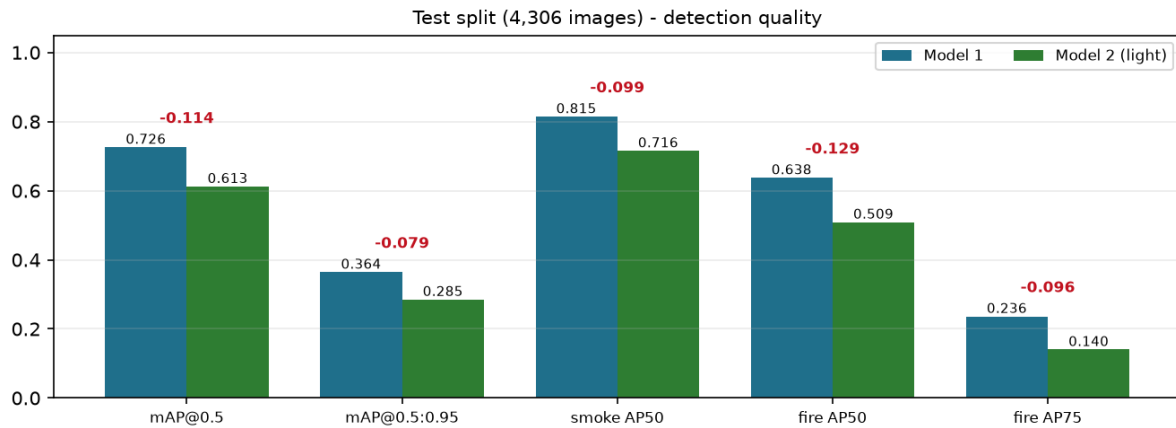


Figure 3 - Detection quality on the test split.

Model 2 is lower on every metric, by roughly 10-13 points of AP50 on each class. Fire - already the harder class for Model 1 - drops the most. Tight localisation (AP75) falls further still, which is consistent with a narrower backbone losing fine spatial detail on small flames.

4. Why the two models need different thresholds

An important detail for anyone deploying Model 2: its confidence scores live on a different scale. FCOS scores a box as the square root of classification confidence times a 'centerness' estimate, and centerness rarely approaches 1, so scores are compressed into a narrow band. On the test split Model 2 has essentially no detections above 0.75: **at Model 1's operating threshold of 0.90, Model 2's recall is exactly zero.**

Comparing the models at the same threshold is therefore meaningless. Each is compared at its own operating point - the lowest threshold that keeps the false-alarm rate under 1%: **0.90 for Model 1 and 0.55 for Model 2.** The live demo tool takes a separate threshold per model for the same reason.

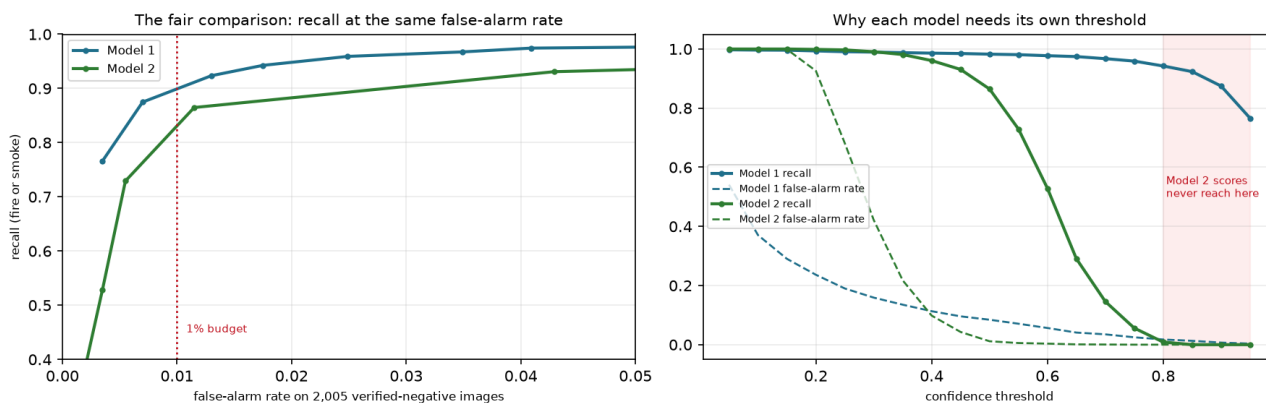


Figure 4 - Left: recall against false-alarm rate, the comparison that matters. Right: the same data against the raw threshold, showing Model 2's compressed score range.

5. Recall at the same false-alarm rate

False-alarm rate	Model 1 recall	Model 2 recall	Gap
0.7% (Model 1's operating point)	0.875	0.763	-11.2 pts
1.0%	0.899	0.831	-6.8 pts
2.0%	0.948	0.883	-6.5 pts

Recall here means an image containing fire or smoke produced at least one box - before temporal confirmation, which recovers some misses on video. Values are interpolated on each model's curve, because the evaluation sweeps thresholds in 0.05 steps and a single step moves Model 2's recall by more than 10 points. The gap narrows as the false-alarm budget loosens.

In plain terms: at Model 1's own operating point, Model 1 finds about 87 of every 100 fire or smoke images and Model 2 about 76. For a safety system, that gap is too large to accept in exchange for speed alone.

6. Speed, size and deployability

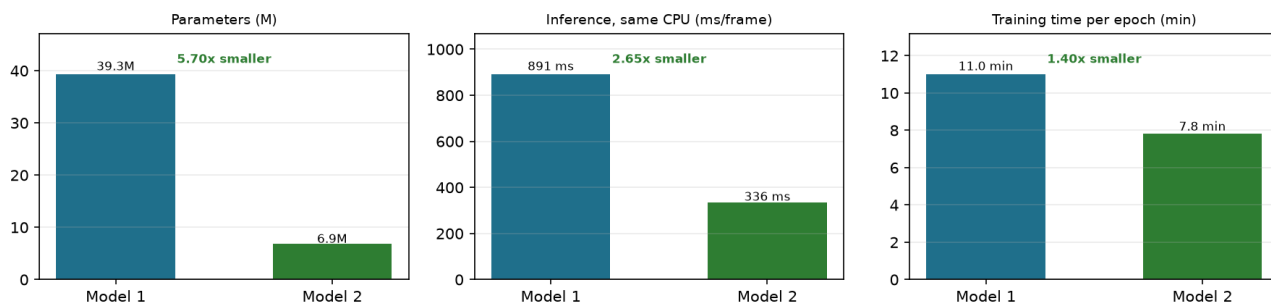


Figure 5 - Cost. Inference measured on the same frames, same CPU, same 640 px input, with the side-by-side comparison tool.

	Model 1	Model 2
ms per frame (CPU)	891	336
Batched ONNX export	fails at batch > 1	verified at batch 1, 4, 8
Path to TensorRT fp16 / multi-camera batching	blocked	open

The CPU figures show the ratio, not deployment speed. The real multi-camera gain comes from batching and TensorRT fp16 on a GPU, which only Model 2 can use; it has not yet been measured on the target GPU, so no camera count is claimed here.

7. Customer-site footage

Both models were run on 54.5 minutes of normal working-day footage from three fixed cameras at an industrial customer site - no fire present, so every alarm is false. Sampling 1 frame per second, alarm on 6 confirmed detections in 15.

Model / threshold	False alarms	Per hour
Model 1 at 0.90 (its operating point)	0	0.0
Model 1 at 0.85	0	0.0
Model 1 at 0.70	3	3.3
Model 2 at 0.55 (its operating point)	0	0.0
Model 2 at 0.50	0	0.0
Model 2 at 0.45	1	1.1

A clean result here is necessary but not sufficient: this footage contains no fire, so it says nothing about recall. Its value is that it is the only real deployment-site data in the project. The false positives it did produce for Model 1, below its threshold, came from a worker's yellow hard hat and from bright dusty haze - confuser classes to add as training negatives.

8. Verdict and next steps

Question	Answer
Is Model 2 faster and smaller?	Yes - 2.65x faster, 5.7x fewer parameters
Can Model 2 be batched for many cameras?	Yes - verified; Model 1 cannot
Is Model 2 as accurate?	No - 0.114 lower mAP@0.5, 11 pts lower recall at 0.7% FPR
Should Model 2 replace Model 1 today?	No. Model 1 stays the accuracy reference.

The result is an honest trade-off, and it is informative rather than disappointing: it shows exactly what deployability costs. Two changes were made at once - a smaller backbone *and* a different head - so the next experiments separate them:

- **Model 2, stage 2** - unfreeze the last two ViT-Ti blocks, as was done for Model 1 (+0.009 there). With only 1.5M trainable parameters, Model 2 is likely to gain more. Cheapest experiment: 15 epochs.
- **ViT-S + FCOS** - Model 1's backbone with Model 2's head. Keeps batched export, runs at roughly Model 1's speed, and shows whether the accuracy loss came from the head or the smaller backbone. If accuracy holds, speed can come from TensorRT and batching instead of a smaller model.
- **Hard negatives from the site** - hard hats, hi-vis and dusty haze as empty-label training images, for whichever model is chosen.
- **Measure on the target GPU** - TensorRT fp16 with batching, before any camera count is stated.

How to present this: Model 1 is the accurate reference; Model 2 demonstrates the deployable architecture and quantifies its accuracy cost. The system is designed for multi-camera scaling; the final model choice and camera capacity will be set by the experiments above and by benchmarking on the target hardware.

Appendix - commands

Model 2 training:

```
python scripts/train_dinov3_detector.py --backbone vit_tiny_patch16_dinov3_qkvb.eupe_lvd1689m --head fcos
--head-convs 2 --fpn-channels 128 --epochs 40 --imgsz 640 --batch 8 --name light_stage1
```

Side-by-side demo, each model at its own operating point:

```
python scripts/compare_models.py --model "Model 1=<model1 best.pt>@0.90" --model "Model 2=<model2
best.pt>@0.55" --video <clip> --max-seconds 120
```

Rebuild this document: `python scripts/build_model_comparison_pdf.py`